

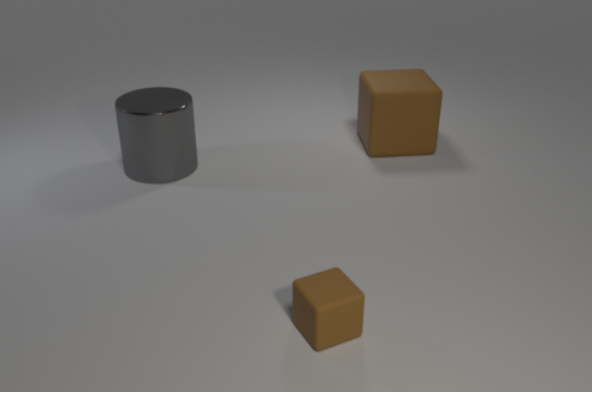


Figure 10. For the above image and the question “What color is the cube behind the cylinder?”, the effective question is “What color is the cube?” (see text for details).

Some pruned questions may be *ill-posed*, meaning that some object references do not refer to a unique object. For example, consider the question “What color is the cube behind the cylinder?”; its associated program is

```
query_color(unique(filter_shape(cube,
relate(behind, unique(filter_shape(cylinder, scene()))))))
```

Imagine executing this program on the scene shown in Figure 10. The innermost *filter_shape* gives a set containing the cylinder, the *relate* returns a set containing just the large cube in the back, the outer *filter_shape* does nothing, and the *query_color* returns *brown*.

This question is not *ill-posed* (Section 3) because the reference to “the cube” cannot be resolved without the rest of the question; however this question’s effective size is less than its actual size because the question can be correctly answered without resolving this object reference correctly.

To compute the effective question, we attempt to prune functions from this program. Starting from the innermost function and working out, whenever we find a function whose input type is *Object* or *ObjectSet*, we construct a pruned question by replacing that function’s input with a *scene* function and executing it. The smallest such pruned program that gives the same answer as the original program is the effective question.

Pruned questions may be ill-posed, so we execute them with modified semantics. The output type of the *unique* function is changed from *Object* to *ObjectSet*, and it simply returns its input set. All functions taking an *Object* input are modified to take an *ObjectSet* input instead by mapping the original function over its input set and flattening the resulting set; thus the *relate* functions return the set of objects in the scene that have the specified relationship with any of the input objects, and the *query* functions return sets of values rather than single values.

Therefore for this example question we consider the fol-

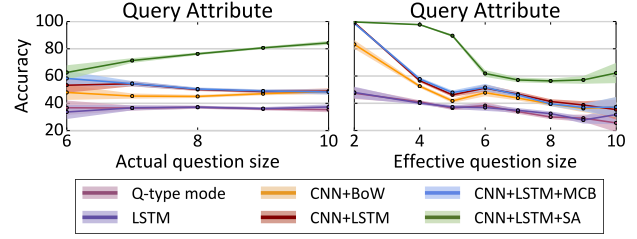


Figure 11. Accuracy on query questions vs. actual and effective question size, restricting to questions with a *same-attribute* relationship. Figure 7 shows the same plots for questions without a *same-attribute* relationship. For both groups of questions we see that accuracy decreases as effective question size increases.

lowing sequence of pruned programs. First we prune the inner *filter_shape* function:

```
query_color(unique(filter_shape(cube,
relate(behind, unique(scene())))))
```

The *relate* function now returns the set of objects which are behind some object, so it returns the large cube and the cylinder (since it is behind the small cube). The *filter_shape* function removes the cylinder, and the *query_color* returns a singleton set containing *brown*.

Next we prune the inner *unique* function:

```
query_color(unique(filter_shape(cube,
relate(behind, scene()))))
```

Since *unique* computes the identity for pruned questions, execution is the same as above.

Next we prune the *relate* function:

```
query_color(unique(filter_shape(cube, scene())))
```

Now the *filter_shape* returns the set of both cubes, but since both are brown the *query_color* still returns a singleton set containing *brown*.

Next we prune the *filter_shape* function:

```
query_color(unique(scene()))
```

Now the *query_color* receives the set of all three input objects, so it returns a set containing *brown* and *gray*, which is different from the original question.

The effective question is therefore:

```
query_color(unique(filter_shape(cube, scene())))
```

B.1. Accuracy vs Question Size

Figure 7 of the main paper shows model accuracy on *query-attribute* questions as a function of actual and effective question size, excluding questions with *same-attribute* relationships. Questions with *same-attribute* relationships have a maximum question size of 10 but questions without *same-attribute* relationships have a maximum size of 20; combining these questions thus leads to unwanted correla-

tions between question size and difficulty.

In Figure 11 we show model accuracy vs. actual and effective question size for questions with same-attribute relationships. Similar to Figure 7, we see that model accuracy either remains constant or increases as actual question size increases, but all models show a clear decrease in accuracy as effective question size increases.

C. Dynamic Module Networks

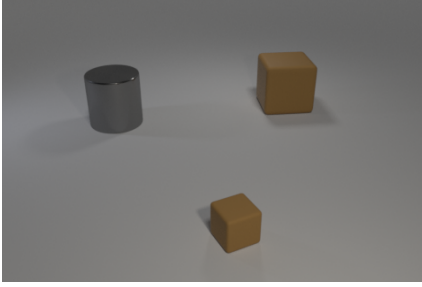
Module networks [2, 3] are a novel approach to visual question answering where a set of differentiable *modules* are used to assemble a custom network architecture to answer each question. Each module is responsible for performing a specific function such as *finding* a particular type of object, *describing* the current object of attention, or performing a *logical and* operation to merge attention masks. This approach seems like a natural fit for the rich, compositional questions in CLEVR; unfortunately we found that parsing heuristics tuned for the VQA dataset did not generalize to the longer, more complex questions in CLEVR.

Dynamic module networks [2] generate network architectures by performing a dependency parse of the question, using a set of heuristics to compute a set of *layout fragments*, combining these fragments to create *candidate layouts*, and ranking the candidate layouts using an MLP.

For some questions, the heuristics are unable to produce any layout fragments; in this case, the system uses a simple default network architecture as a fallback for answering that question. On a random sample of 10,000 questions from the VQA dataset [4], we found that dynamic module networks resorted to default architecture for 7.8% of questions; on a random sample of 10,000 questions from CLEVR, the default network architecture was used for 28.9% of questions. This suggests that the same parsing heuristics used for VQA do not apply to the questions in CLEVR; therefore the method of [2] did not work out-of-the box on CLEVR.

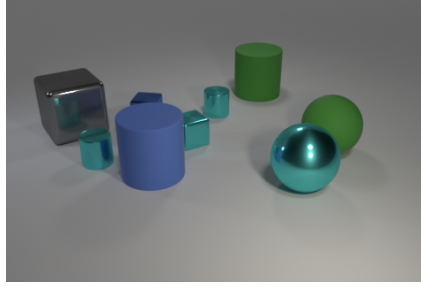
D. Example images and questions

The remaining pages show randomly selected images and questions from CLEVR. Each question is annotated with its answer, question type, and size. Recall from Section 3 that a question’s *type* is the outermost function in the question’s functional program, and a question’s *size* is the number of functions in its program.



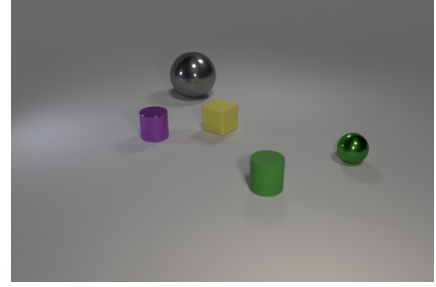
Q: There is a rubber cube in front of the big cylinder in front of the big brown matte thing; what is its size?
A: small
Q-type: query_size
Size: 14

Q: What color is the object that is on the left side of the small rubber thing?
A: gray
Q-type: query_color
Size: 7



Q: Are there fewer metallic objects that are on the left side of the large cube than cylinders to the left of the cyan shiny block?
A: yes
Q-type: less_than
Size: 16

Q: There is a green rubber thing that is left of the rubber thing that is right of the rubber cylinder behind the gray shiny block; what is its size?
A: large
Q-type: query_size
Size: 17



Q: Is the number of tiny metal objects left of the purple metallic cylinder the same as the number of small metal objects to the left of the small metal sphere?
A: no
Q-type: equal_integer
Size: 19

Q: Do the large shiny object and the thing to the left of the big gray object have the same shape?
A: no
Q-type: equal_shape
Size: 13

Q: There is another cube that is made of the same material as the small brown block; what is its size?
A: large
Q-type: query_size
Size: 9

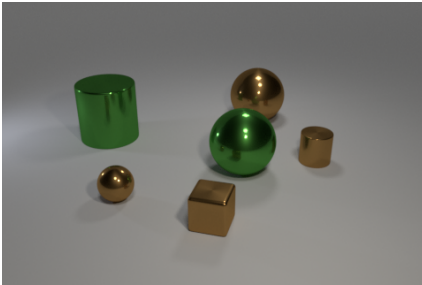
Q: What number of other matte objects are the same shape as the small rubber object?
A: 1
Q-type: count
Size: 7

Q: What is the size of the matte thing that is on the left side of the large cyan object and in front of the small blue thing?
A: large
Q-type: query_size
Size: 14

Q: The green matte thing that is behind the large green thing that is to the right of the big cyan metal object is what shape?
A: cylinder
Q-type: query_shape
Size: 14

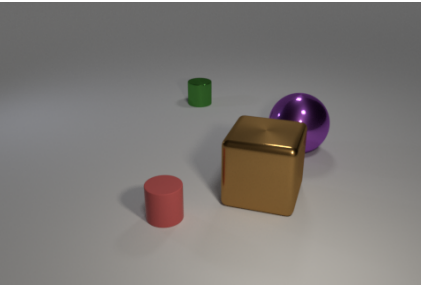
Q: There is a small thing that is the same color as the tiny shiny ball; what is it made of?
A: rubber
Q-type: query_material
Size: 9

Q: Is there anything else that is the same shape as the tiny yellow matte thing?
A: no
Q-type: exist
Size: 7



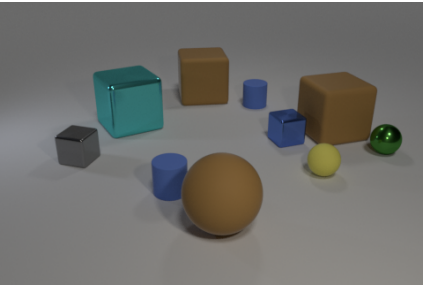
Q: What color is the small shiny cube?
A: brown
Q-type: query_color
Size: 6

Q: There is a tiny shiny sphere left of the cylinder in front of the large cylinder; what color is it?
A: brown
Q-type: query_color
Size: 13



Q: What number of cylinders are purple metal objects or purple matte things?
A: 0
Q-type: count
Size: 9

Q: Does the large purple shiny object have the same shape as the tiny object that is behind the matte thing?
A: no
Q-type: equal_shape
Size: 14



Q: There is a large brown block in front of the tiny rubber cylinder that is behind the cyan block; are there any big cyan metallic cubes that are to the left of it?
A: yes
Q-type: exist
Size: 20

Q: There is a big shiny object; are there any blue shiny cubes behind it?
A: no
Q-type: exist
Size: 9

Q: Is there a tiny thing that has the same material as the brown cylinder?
A: yes
Q-type: exist
Size: 7

Q: What is the material of the tiny object to the right of the brown shiny ball behind the tiny shiny cylinder?
A: metal
Q-type: query_material
Size: 14

Q: There is an object that is both on the left side of the brown metal block and in front of the large purple shiny ball; how big is it?
A: small
Q-type: query_size
Size: 16

Q: There is a big purple shiny object; what shape is it?
A: sphere
Q-type: query_shape
Size: 6

Q: What number of cylinders have the same color as the metal ball?
A: 0
Q-type: count
Size: 7

Q: The cyan block that is the same material as the tiny gray thing is what size?
A: large
Q-type: query_size
Size: 9